Software Effort Estimator

Technical Documentation

How the Model Works — From Raw Inputs to Explainable Predictions

METHODOLOGY	FAHP (Chang 1996) + Weighted-Kernel SVR (Sehra et al. 2018) + SHAP
TRAINING DATA	COCOMO-81 (63 projects) · GitHub open-source (91 repos)
VALIDATION	Nested 10-fold outer / 5-fold inner CV · popularity hold-out · NASA-93 external test
HEADLINE MMRE	0.3910 (COCOMO-81) · 0.2055 (GitHub)
RESEARCH CONTEXT	Mitacs Accelerate Internship — Industry partner model deliverable

Keshav · Mitacs Research Demo · 2026

1. What Is Software Effort Estimation?

Software effort estimation answers one question: **how many person-months will it take to build this project?** Getting this right matters enormously. Underestimate and the team burns out trying to meet an impossible deadline. Overestimate and resources sit idle while competitors ship faster.

For decades, companies relied on rules of thumb, gut feel, or simple spreadsheets. The COCOMO-81 model (Boehm, 1981) was a landmark — the first systematic, empirically grounded approach. It said: *if you tell me the size of the codebase and 15 project attributes, I can give you a reliable effort estimate.*

This tool builds on that foundation and extends it in three ways:

- **Better feature weighting** — Fuzzy AHP (FAHP) assigns each cost driver a mathematically derived importance score, so the model knows which factors matter most for your type of project.
- **Non-linear learning** — Support Vector Regression (SVR) with a weighted kernel learns patterns from historical data and is evaluated separately from the original multiplicative COCOMO formula.
- **Explainability** — SHAP values break down every prediction into per-feature contributions in person-months, so you know *why* the estimate is what it is.

> Think of it like this: COCOMO is a recipe from 1981 written for defence projects. This tool is the same recipe, upgraded with a modern oven, better ingredients, and a chef who explains every step.

1.1 The Core Metric: MMRE

Throughout this document you will see **MMRE** (Mean Magnitude of Relative Error). It is the standard accuracy measure for effort estimation:

$$\text{MMRE} = \frac{1}{n} \times \Sigma \left| \text{actual_effort} - \text{predicted_effort} \right| / \text{actual_effort}$$

An MMRE of 0.30 means predictions are, on average, 30% away from the true value. Lower is better. PRED(25) is the complementary metric: the fraction of predictions within 25% of the true value. Higher is better.

MMRE	What it means	Context
< 0.10	Excellent — within 10% on average	Rarely achieved on real projects
0.10 – 0.25	Lower relative error in this sample	GitHub nested-CV result: 0.2055
0.25 – 0.40	Moderate relative error in this sample	COCOMO nested-CV result: 0.3910
> 0.50	Poor — coin-flip territory	Typical spreadsheet estimation

2. The COCOMO-81 Foundation

COCOMO stands for COnstructive COst MOdel. Barry Boehm published it in 1981 after analysing 63 software projects at TRW Aerospace. The fundamental equation is:

$$\text{Effort (PM)} = a \times \text{KLOC}^b \times \text{EAF}$$

Where:

- **KLOC** = thousands of lines of code (project size)
- **a, b** = constants that depend on development mode (Organic / Semi-detached / Embedded)
- **EAF** = Effort Adjustment Factor — a product of 15 cost driver multipliers

2.1 Development Modes

Mode	Description	a	b	Typical project
Organic	Small team, familiar domain, flexible requirements	2.4	1.05	Internal tools, small web apps
Semi-detached	Mixed experience, moderate constraints	3.0	1.12	Mid-size commercial software
Embedded	Hard real-time, safety-critical, tight hardware constraints	3.6	1.20	Avionics, medical devices

2.2 The 15 Cost Drivers (EAF)

Each cost driver is rated on a 5-point scale: Very Low, Low, Nominal, High, Very High. Each rating maps to a multiplier. The product of all 15 multipliers is the EAF. EAF < 1.0 means the project conditions are **favourable** (reducing effort); EAF > 1.0 means they are **challenging** (increasing effort).

Category	Driver	What it measures	VL → VH range
Product	RELY	Consequence of failure (inconvenience → safety-critical)	0.75 → 1.40
Product	DATA	Database and data complexity	0.94 → 1.16
Product	CPLX	Algorithmic and architectural complexity	0.70 → 1.30
Computer	TIME	CPU execution-time constraint	1.00 → 1.30
Computer	STOR	Memory constraint	1.00 → 1.21
Computer	VIRT	Platform/VM volatility	0.87 → 1.30
Computer	TURN	Development turnaround time	0.87 → 1.15
Personnel	ACAP	Analyst/architect capability	1.46 → 0.71
Personnel	AEXP	Application domain experience	1.29 → 0.82

Category	Driver	What it measures	VL → VH range
Personnel	PCAP	Programmer capability	1.42 → 0.70
Personnel	VEXP	Virtual machine experience	1.21 → 0.87
Personnel	LEXP	Programming language experience	1.14 → 0.95
Project	MODP	Modern programming practices	1.24 → 0.82
Project	TOOL	Software tools quality	1.24 → 0.83
Project	SCED	Schedule constraint	1.23 → 1.10

Example: A 50 KLOC Organic project with all Nominal ratings gives:
Effort = 2.4 × 50^1.05 × 1.0 = 2.4 × 60.8 = **~146 person-months**
If the team has Very High programmer capability (PCAP multiplier = 0.70) and High complexity (CPLX = 1.15):
EAF = 0.70 × 1.15 = 0.805 → Effort = 146 × 0.805 = **~118 person-months**

3. Fuzzy Analytic Hierarchy Process (FAHP)

COCOMO treats all 15 cost drivers as equally important to the prediction task — it just multiplies them all together. But in reality, some features matter much more than others. **FAHP is how we discover and encode that importance.**

3.1 What Is the Analytic Hierarchy Process?

AHP (Saaty, 1980) is a decision-making framework that assigns weights to criteria by asking: *how much more important is criterion A than criterion B?* You build a matrix of pairwise comparisons and extract weights mathematically.

The problem with standard AHP is that human judgements are imprecise. Saying "CPLX is 3× more important than LEXP" is hard to justify exactly. **Fuzzy AHP** replaces exact numbers with triangular fuzzy numbers (TFNs) that capture this uncertainty.

3.2 Triangular Fuzzy Numbers

A TFN is written as (l, m, u): a lower bound, a most likely value, and an upper bound. Instead of saying "A is exactly 3× more important than B", we say "A is between 1× and 5× more important, most likely 3×" — written as (1, 3, 5).

$$\text{TFN} = (l, m, u) \text{ where } l \leq m \leq u$$

The TFN scale used in this model (Buckley 1985 / Chang 1996):

Linguistic judgement	TFN
Equally important	(1, 1, 1)
Weakly more important	(1, 2, 3)
Moderately more important	(1, 3, 5)
Strongly more important	(3, 5, 7)
Very strongly more important	(5, 7, 9)
Absolutely more important	(7, 9, 9)

3.3 Chang's Extent Analysis (1996)

This model uses Chang's extent analysis to derive feature weights from the pairwise comparison matrices. The algorithm has three steps:

1. **Step 1 — Build pairwise matrices:** For each pair of features (i, j), compute the importance ratio r = corr(feature_i, effort) / corr(feature_j, effort) using Spearman correlations with the training data. Convert r to a TFN. Build an n×n matrix of TFNs.
2. **Step 2 — Synthetic extent values:** For each feature i, sum across all j to get a fuzzy row sum S_i = (l_i, m_i, u_i). Normalise to get the synthetic extent value.

$$S_i = (\Sigma_j l_{ij} / \Sigma_i \Sigma_j u_{ij} \, , \, \Sigma_j m_{ij} / \Sigma_i \Sigma_j m_{ij} \, , \, \Sigma_j u_{ij} / \Sigma_i \Sigma_j l_{ij})$$

3. **Step 3 — Degree of possibility:** For each feature i, compute V(S_i ≥ S_j) for all j ≠ i. The weight of feature i is the minimum of these values, then normalise.

$$V(S_i \geq S_j) = 1 \text{ if } m_i \geq m_j,\ 0 \text{ if } l_j \geq u_i,\ \text{else } (l_j - u_i) / ((m_i - u_i) - (m_j - l_j))$$

$$\text{weight}_i = \min_{j \neq i} V(S_i \geq S_j) \rightarrow \text{normalise so } \Sigma \text{ weights} = 1$$

3.4 Worked Example: 3 Features

Suppose we have 3 features with Spearman correlations with effort: CPLX (r=0.65), ACAP (r=0.50), LEXP (r=0.20). We build a 3×3 pairwise matrix:

	CPLX	ACAP	LEXP
CPLX	(1,1,1)	(1,2,3)	(3,5,7)
ACAP	(1/3,1/2,1)	(1,1,1)	(1,3,5)
LEXP	(1/7,1/5,1/3)	(1/5,1/3,1)	(1,1,1)

After running Chang's extent analysis, we might get weights like:

Feature	FAHP weight	Interpretation
CPLX	0.52	Most important — dominates prediction
ACAP	0.35	Moderately important
LEXP	0.13	Less important

Key insight: FAHP doesn't just rank features — it produces weights that are embedded directly into the SVR kernel. A feature with weight 0.52 will have its distances measured ~4× more carefully than one with weight 0.13. Whether this lowers prediction error is measured by the matched weighted/unweighted nested-validation comparison.

4. Weighted-Kernel Support Vector Regression (SVR)

Once FAHP gives us feature weights, we need a model that can learn non-linear patterns from the 63 COCOMO-81 training projects. That model is SVR.

4.1 What Is SVR?

Support Vector Regression learns a function that predicts continuous outputs (like person-months). Unlike linear regression, it uses a **kernel function** to measure similarity between projects in a high-dimensional feature space, allowing it to capture non-linear relationships.

The key idea: instead of minimising the sum of squared errors (like OLS regression), SVR tries to find a function where most predictions fall within an ε-tube around the true values. Points outside the tube are penalised by a cost parameter C.

$$\text{SVR finds } f(x) \text{ such that } |y_i - f(x_i)| \leq \varepsilon \text{ for most training points}$$

4.2 The RBF Kernel

This model uses the Radial Basis Function (RBF) kernel, which measures how "close" two projects are in feature space:

$$K(x_i, x_j) = \exp(-\gamma \times ||x_i - x_j||^2)$$

Two projects that are very similar (small distance) have K ≈ 1. Two projects that are very different have K ≈ 0. The model learns that similar projects should have similar effort estimates. γ controls how quickly similarity falls off with distance — a hyperparameter tuned by cross-validation.

4.3 The FAHP Weighting — The Core Innovation

Here is where FAHP and SVR connect. Instead of measuring distance equally across all features, we **stretch the feature space** by the FAHP weights. Features that matter more are measured more carefully:

$$K_FAHP(x_i, x_j) = \exp(-\gamma \times ||\sqrt{\theta} \cdot (x_i - x_j)||^2)$$

Where θ is the vector of FAHP weights and √θ is applied element-wise. This is equivalent to:

$$x_weighted = x \times \sqrt{\theta}$$

Concretely: if ACAP has FAHP weight 0.35 and LEXP has weight 0.05, then the weight ratio is 0.35/0.05 = 7.0×. Because the kernel uses *squared* distance, the effective distance scaling factor is √7.0 = 2.6× — meaning a difference in ACAP is measured 2.6× more sensitively in distance terms than the same numerical difference in LEXP. The kernel "pays more attention" to the features that matter.

4.4 The Full Training Pipeline

Step	What happens	Why
1. Log transform	x → log(x), y → log(1 + y)	Effort is right-skewed; log space makes it more symmetric
2. Scale	MinMaxScaler to [0, 1]	SVR is sensitive to feature scale differences
3. FAHP weight	x_weighted = x_scaled × √(FAHP_weights)	Embeds feature importance into kernel distance
4. SVR fit	SVR(RBF, C, γ, ε).fit(X_weighted, y_scaled)	Learn non-linear effort surface
5. Predict	Predict → inverse-scale → expm1	Convert back from log-space to person-months

4.5 Hyperparameter Tuning

Three SVR hyperparameters are tuned by exhaustive grid search, using five-fold inner-CV MMRE as the objective inside each outer-training partition:

- **C** (cost): penalty for predictions outside the ε-tube. Higher C = tighter fit, risk of overfit.
- **γ** (gamma): how quickly the RBF kernel falls off. Higher γ = more local decisions.
- **ε** (epsilon): the width of the no-penalty tube. Larger ε = more tolerance.

Why nested validation? A single resampling loop cannot both choose hyperparameters and provide an untouched performance estimate. Ten outer folds are used only for evaluation; within each outer-training partition, five inner folds select C, gamma and epsilon. Scaling and FAHP weights are also fitted only on the relevant training rows. Deployment parameters are selected separately after evaluation.

5. SHAP Explainability

SHAP (SHapley Additive exPlanations, Lundberg & Lee 2017) answers: *"The model predicted 180 person-months — how much did each feature contribute?"* This matters because a number without a reason is useless for planning.

5.1 The Shapley Value Concept

Shapley values come from cooperative game theory. Imagine each feature is a "player" contributing to the prediction "game". The Shapley value of a feature is its **average marginal contribution** across all possible orderings of features.

In plain English: SHAP asks "if we added feature X to the model after considering all possible subsets of other features, what would it add to the prediction on average?" This gives a fair, mathematically rigorous attribution.

$$\text{prediction} = \text{base_value} + \Sigma\ \text{SHAP_value}(\text{feature_i})$$

The base value is the model's average prediction across all training projects. Each SHAP value is in the same units as the prediction (person-months), so you can say: "High CPLX added +23 PM above the baseline."

5.2 KernelExplainer for SVR

SVR doesn't have a closed-form SHAP solution like tree models do. This tool uses **KernelExplainer** (model-agnostic SHAP), which:

- Samples 30 background projects from the training set
- Evaluates the full pipeline (log → scale → weight → SVR → inverse-scale → expm1) on 150 random feature coalitions
- Fits a weighted linear model to approximate SHAP values for the current project

The SHAP values are computed in the **original raw feature space**, so each value directly represents person-month impact — not some abstract scaled quantity.

5.3 Reading SHAP Outputs

Chart	What to read
Feature Contributions bar	Red bars = features pushing effort UP above baseline. Blue bars = features pushing effort DOWN. Bar length = magnitude in PM.
Waterfall chart	Shows the cumulative path from baseline (avg PM) to your prediction. Each step adds or subtracts PM.

Chart	What to read
FAHP vs SHAP bar chart	Compares the prior weights (what FAHP thought mattered) vs posterior importance (what the trained model actually learned). Divergence is a research finding.

Note on the baseline value: The SHAP base value is the mean model prediction over the background dataset. This tool uses the 50 training projects with effort ≤ 600 PM (80th percentile) as the representative background (seed=42, stable across runs). The 13 excluded projects are 1980s defence mega-programs (up to 11,400 PM) whose inclusion would push the base to 620 PM — making every modern-project waterfall open at an unintuitive starting point 4–6× the final prediction. The representative background gives a base of **~116 PM**, close to the geometric mean of the training distribution (127 PM). SHAP values remain in person-months throughout; additivity verified at 0.00% error. See Section 9 for a worked example with real output.

Important caveat — Nominal ratings can still show large SHAP values. SHAP is always relative to the *training mean*, not to a "zero effort" baseline. If the 63 COCOMO-81 training projects averaged High TIME constraints, then a project with Nominal TIME will show a *negative* SHAP value — not because Nominal TIME reduces effort in absolute terms, but because it is below the training average. Similarly, Dev Mode encoded as an ordinal can absorb large SHAP values if it is the strongest separator in the training data. Always read SHAP values as "relative to what the model typically sees", not as absolute effort drivers.

6. GenAI Productivity Adjustment

COCOMO-81 was published in 1981. Its TOOL cost driver captures the quality of development tools — and at Very High, it reduces effort by at most 17%. That was the best tools could do in 1981. GitHub Copilot was released in 2022.

6.1 The Research Evidence

Study	n	Finding	Effort impact
Peng et al. (2023) — GitHub	95 developers	Copilot: 55% faster on isolated coding task (controlled experiment) — full-project gain not measured by this study	Task-level: −55%
McKinsey Global Institute (2023)	850 projects	20–45% productivity gain depending on AI adoption depth (industry report)	−20–45%
Kalliamvakou (2022) — GitHub blog	Copilot cohort	Copilot users merged PRs 26% faster, reported higher satisfaction	~−26% on PRs
COCOMO-81 TOOL driver	63 projects	Very High tools: maximum 17% effort reduction	max −17%

The research gap: No existing effort estimation model — including Sehra et al. (2018), which this tool implements — accounts for modern AI coding tools. COCOMO caps at 17%. Studies show 22–55%. Quantifying this gap empirically on your company's data is the novel Mitacs research contribution.

6.2 How the Adjustment Works

The GenAI adjustment is a **post-hoc multiplier** applied to the SVR prediction. It is not baked into the model — it is applied on top, transparently:

$$AI_effort = baseline_effort \times AI_multiplier$$

AI level	Multiplier (central)	Range	Source
None — no AI tools	1.00	—	—
Some — Copilot / ChatGPT	0.78	0.75–0.82	Peng et al. 2023, McKinsey 2023
Heavy — Cursor / AI-first team	0.62	0.55–0.70	McKinsey 2023, Kalliamvakou 2022

The range is shown as a confidence band in the UI, reflecting the study variation. A "Heavy AI" team of 50 that would have needed 200 PM is estimated at 200 × 0.62 = **124 PM**, saving 76 PM (38%). The 80% prediction interval propagates through: 200 × 0.55 = 110 PM to 200 × 0.70 = 140 PM.

Important caveat: These multipliers come from task-level studies and enterprise surveys, not from effort estimation research. Applying them to full-project effort estimates involves uncertainty. The central estimate is the research midpoint; real impact depends on your codebase, team adoption rate, and task type.

7. The GitHub-Trained Model

The COCOMO-81 model was trained on 1980s defense software. To show the methodology works on **modern software projects**, we collected 91 real GitHub repositories and trained the same FAHP + SVR pipeline on their data.

7.1 Features

Feature	What it measures	Why it matters
Contributors	Unique people who committed	More people = more coordination overhead (Brooks's Law)
PR Review Time	Avg days from PR open to merge	Slow reviews = less capable / slower team
PR Merge Rate	Fraction of PRs merged	Low merge rate = rework and wasted effort

Feature	What it measures	Why it matters
Test File Ratio	Fraction of files that are tests	High test coverage = mature engineering practices
Language Count	Number of programming languages	More languages = more context-switching overhead
Commit Frequency	Commits per week	Active development intensity signal
Contributors × Freq	Interaction term	Large fast-moving teams — captures combined scale
AI Tool Adoption	Number of AI tools configured	Copilot, Cursor, Claude Code, Agents (0–3)

7.2 The Effort Proxy

GitHub doesn't have Jira tickets with actual hours. Instead, effort is approximated as:

$$\text{author_months} = \text{unique (author, date) pairs across all commits} / 20$$

A contributor who commits 10 times in one day still counts as 1 author-day. Dividing by 20 converts working days to months (approx. 20 working days/month). This is an imperfect proxy — it measures developer presence, not hours of productive work — but it correlates with real project size and effort across the dataset.

Sampling ceiling: collection is capped at 1,200 commits per repository, approximately 60 author-months under this proxy. Predictions above that cap are extrapolations and have not been validated.

7.3 Transparency: Feature/Target Overlap

Since contributor_count, the interaction term, and author_months all derive from the same commit-activity log, a reviewer should ask whether the low MMRE reflects genuine predictive signal or shared measurement source. We checked all three key correlations:

Feature	Spearman r with author_months	Note
contributor_count	+0.705	High — both derived from who committed
contributors_x_freq	+0.650	High — interaction amplifies the overlap
commit_frequency	+0.395	Moderate

This is **construct overlap, not train/test leakage.** Strict leakage means test-set information bleeding into training, inflating CV scores — the CV here is clean. What exists is that author_months deduplicates to unique person-days and divides by 20, so it is strongly related to — but not a deterministic function of — contributor count. A 50-person team each committing once total has very different author_months than 50 people each active for months. The r = 0.70 says they are strongly correlated, not identical.

The low MMRE therefore reflects a genuine team-size → effort signal that cannot be cleanly separated from shared measurement source. Jira-tracked hours as an independent ground truth would remove the overlap and give a clean accuracy figure.

> An earlier version of this section reported Spearman(commit_frequency, author_months) = 0.04 and concluded "no meaningful leakage exists." That figure was computed with Pearson correlation, not Spearman. The corrected Spearman values are shown in the table above. The conclusion changes: construct overlap exists in the dominant feature (r = 0.70), and the MMRE should be interpreted accordingly — as a strong proof-of-concept, not as a clean accuracy benchmark.

7.4 Popularity Hold-out (Not Temporal Validation)

The current CSV has no trustworthy project-date field, so a chronological test cannot be claimed. The available sensitivity check sorts repositories by star count. Stars measure popularity, not time:

- Train on the higher-star partition
- Test on the lower-star partition
- Select hyperparameters with inner five-fold CV on the training partition only
- Re-fit scalers and FAHP weights entirely on training data

Validation method	MMRE	PRED(25)	Notes
Nested 10x5 CV	0.2055	70.3%	Headline GitHub estimate
Popularity hold-out	0.1992	64.3%	High-star train to low-star test (n=28)
COCOMO-81 nested CV	0.3910	46.0%	Separate historical benchmark

> This result is a popularity sensitivity check only. It must not be described as temporal generalisation. Future collection records repository creation and sampled commit dates so a genuinely chronological evaluation can be performed.

8. Validation Results

8.1 Ablation Study — Does FAHP Actually Help?

A fair ablation must give the unweighted baseline its OWN hyperparameter search. Reusing the C/gamma/epsilon tuned for the FAHP-weighted feature space cripples the baseline and inflates the apparent benefit. Below, each variant is independently tuned over the same grid, with leakage-free nested evaluation:

Model variant	Nested-CV MMRE	FAHP comparison
Plain SVR (no FAHP, independently tuned)	0.4312	baseline

Model variant	Nested-CV MMRE	FAHP comparison
FAHP-Weighted SVR	0.3910	9.3% lower MMRE in this comparison

In matched nested validation, FAHP weighting changes COCOMO-81 MMRE by **9.3%** (0.4312 to 0.3910). This is a sample-specific comparison, not proof that weighting improves every future dataset.

GitHub comparison: the independently tuned unweighted model scores 0.2149, while the weighted model scores 0.2055, a 4.4% lower MMRE in these nested folds. The result is reported as a comparison because it does not establish a universal causal improvement from FAHP.

8.2 Cross-Dataset Generalisation (NASA-93)

To test whether the COCOMO-81 model generalises beyond its training distribution, we evaluate it on NASA-93 — 93 entirely different NASA software projects collected 12 years later. No NASA-93 data was used in training or tuning.

Evaluation	Dataset	n	MMRE	PRED(25)
Nested 10x5 CV	COCOMO-81	63	0.3910	46.0%
External test	NASA-93	93	0.5175	46.2%

MMRE is 0.3910 in nested COCOMO-81 validation and 0.5175 on NASA-93. The external result measures transfer to a separate dataset; it does not guarantee performance for modern commercial projects.

8.3 The 80% Prediction Interval

Beyond point estimates, the tool shows an 80% prediction interval — a range where approximately 4 out of 5 real projects should fall. This is computed empirically from pure nested outer-fold residuals (no parametric assumption):

```
error = (prediction - actual) / actual
PI = [prediction / (1 + q90), prediction / (1 + q10)]
```

For COCOMO-81, q10=-0.443 and q90=+0.680. If the point prediction is 100 PM, the transformed 80% interval is [59.5, 179.5] PM. Roughly one in five comparable projects may fall outside this empirical range.

8.4 Per Development Mode Breakdown

Not all projects are equally predictable. Splitting the COCOMO-81 results by development mode reveals an interesting pattern:

Mode	n	Nested-fold MMRE	PRED(25)	Notes
Organic	24	0.3317	54.2%	Small familiar projects
Semi-detached	11	0.3757	27.3%	Mixed experience
Embedded	28	0.4479	46.4%	Hard real-time or constrained

These subsets are small, especially semi-detached (n=11). Their metrics are descriptive breakdowns of the outer-fold predictions, not independent validation studies.

Takeaway: Use per-mode numbers as context, not a strict trust ranking. Sampling variation can be substantial at these group sizes.

9. End-to-End Worked Example

Let's walk through a complete prediction for a realistic project: a **FinTech payment processing API**, 40 KLOC, Semi-detached mode, with a moderately experienced team, some schedule pressure, and Copilot adopted.

Step 1 — Gather Inputs

Input	Value	Rationale
LOC	40 KLOC	Mid-size API service
RELY	High (1.15)	Financial transactions — SLA breach = money loss
DATA	Nominal (1.00)	Standard database complexity
CPLX	High (1.15)	Real-time settlement logic
TIME	Nominal (1.00)	Standard latency requirements
STOR	Nominal (1.00)	Cloud deployment, no memory pressure
VIRT	Low (0.87)	Stable cloud infrastructure
TURN	Low (0.87)	Fast CI/CD pipeline
ACAP	High (0.86)	Strong architects
AEXP	Nominal (1.00)	Some FinTech experience
PCAP	High (0.86)	Solid developer team
VEXP	High (0.90)	AWS-experienced team
LEXP	Nominal (1.00)	TypeScript / Node.js — familiar
MODP	High (0.91)	Good Agile practices

Input	Value	Rationale
TOOL	High (0.91)	Copilot + good tooling
SCED	High (1.04)	Fixed product launch date
Dev Mode	Semi-detached	Mixed team, moderate constraints

Step 2 — Compute the Raw COCOMO Estimate

EAF = 1.15 × 1.00 × 1.15 × 1.00 × 1.00 × 0.87 × 0.87 × 0.86 × 1.00 × 0.86 × 0.90 × 1.00 × 0.91 × 0.91 × 1.04
EAF ≈ 0.574 (favourable team/tools conditions reducing the baseline)
Baseline (Semi-det) = 3.0 × 40^1.12 = 3.0 × 62.3 = 186.9 PM COCOMO Effort = 186.9 × 0.574 ≈ 107 PM

Step 3 — SVR Prediction (What the Model Actually Returns)

The SVR doesn't use the COCOMO formula directly. It:

- Converts the raw inputs (LOC, 15 multipliers, dev mode ordinal) to a feature vector
- Log-transforms, MinMax-scales, and FAHP-weights the vector
- Passes it through the trained RBF kernel SVR
- Inverse-scales and expm1 transforms back to person-months

For these exact inputs the model returns **111 PM** — nearly identical to the COCOMO formula's 107 PM. This is expected: with a well-calibrated EAF (0.574) on a Semi-detached project squarely within the training distribution, the SVR and the COCOMO multiplicative formula converge. The SVR adds value on projects where non-linear interactions between drivers matter more — here, the drivers are moderate and the two methods agree.

Step 4 — SHAP Decomposition (real model output, seed=42, representative background n=50)

Background: the 50 training projects with effort ≤ 600 PM (the 80th percentile). The 13 excluded projects are 1980s defence mega-programs (up to 11,400 PM) that skew the arithmetic mean to 620 PM — not representative of modern commercial software. The resulting base value is **115.8 PM**, close to the geometric mean (127 PM), and the SHAP values are in PM units throughout. Additivity verified: 0.00% error.

Feature	SHAP value	Reading
LOC (40 KLOC)	+69.7 PM	40 KLOC is larger than the typical background project — main effort driver

Feature	SHAP value	Reading
RELY (High)	+14.0 PM	Financial stakes push above baseline
DevMode (Semi)	+7.6 PM	Semi-detached slightly above the mixed background average
TOOL (High)	−23.5 PM	Good tooling pulls effort down
TIME (Nominal)	−21.8 PM	No tight latency SLA — typical background has more pressure
TURN (Low)	−15.1 PM	Fast CI/CD reduces friction
PCAP (High)	−12.4 PM	Strong developers are faster than background average
VEXP (High)	−11.3 PM	Cloud-experienced team
ACAP (High)	−7.7 PM	Strong architects reduce effort
MODP (High)	−4.6 PM	Good engineering practices
All others	0.0 PM	DATA, CPLX, STOR, VIRT, AEXP, LEXP, SCED — negligible vs background
Baseline	115.8 PM	Mean prediction over 50 representative training projects (seed=42, stable)
Total	**110.8 PM**	115.8 − 5.0 = 110.8 · additivity: 0.00% error

Step 5 — Schedule and Team

Calendar months = 2.5 × 111^0.38 = 2.5 × 5.98 ≈ 15 months
Recommended team = 111 / 15 ≈ 7 engineers

Step 6 — GenAI Adjustment (Copilot adopted)

AI effort = 111 × 0.78 = 87 PM (22% reduction)
AI calendar = 2.5 × 87^0.38 = 2.5 × 5.46 ≈ 14 months | AI team = 6 engineers
80% PI: 111 / (1 + +0.680) = 66 PM to 111 / (1 + -0.443) = 199 PM

Summary for the FinTech API project (illustrative model output):
SVR prediction: 110.8 PM · COCOMO: 107 PM · Models converge (well-calibrated inputs)
Without AI: 111 PM, 15 months, 7 engineers
With Copilot: 87 PM, 14 months, 6 engineers (saves ~24 PM = 22%)
80% empirical interval: 66–199 PM (wide — reflects genuine estimation uncertainty)
SHAP baseline: 115.8 PM (mean over 50 representative training projects, seed=42)
Top effort driver: LOC (+69.7 PM); top reducers: TOOL (−23.5 PM), TIME (−21.8 PM)

10. Limitations and Scope of Validity

No model is perfect. Being transparent about limitations is a sign of rigorous research, not weakness. Use these estimates as structured guidance, not budget commitments.

10.1 COCOMO-81 Model

- **Training data age:** 63 projects from the 1980s, mostly defense/aerospace. The model has never seen React, microservices, CI/CD, or cloud-native development.
- **LOC as a proxy:** Modern teams often work with story points, not LOC. The model accepts LOC as a proxy for scope — use sprint velocity × total sprints as an approximation.
- **Solver differences:** Sehra et al. (2018) used LSSVM; this implementation uses sklearn SVR. The weighted-kernel formulation is faithful; the solver is an acknowledged simplification.
- **FAHP weight derivation:** Expert pairwise comparisons are replaced with Spearman correlation ratios — a data-driven proxy. In a full implementation, domain experts would provide the comparisons directly.
- **Size-response compression.** The model's effort response to project size is slightly sublinear — the effective in-range exponent is ~0.88, versus COCOMO-81's 1.05 — a consequence of RBF-kernel smoothing pulling predictions toward the training distribution. Within the training range this is minor, but the compression intensifies with distance from the data: beyond the largest training projects it produces systematic under-prediction (the basis for the extrapolation warning at 73 KLOC in Organic mode). Large-project estimates should therefore be read as lower bounds, and inputs beyond the per-mode training range treated as unreliable rather than merely uncertain.

10.2 GitHub Model

- **Effort proxy quality:** Author-months from commit logs ≠ actual billed hours. Open-source volunteers often commit in bursts; paid developers have more consistent patterns.
- **Sampling cap:** At most 1,200 commits are sampled per repository (approximately 60 author-months). Predictions above that range are extrapolated and not validated.
- **Open-source bias:** GitHub repos are maintained by volunteers with different motivations than commercial development teams. Coordination overhead and rework patterns may differ.
- **Feature availability:** AI tool adoption is inferred from config files — not all teams configure their tools in discoverable ways, leading to underestimation of adoption.

10.3 GenAI Multipliers

- **Post-hoc adjustment:** Multipliers are applied to total effort. AI tools primarily help with coding-specific tasks (code completion, test generation); planning, stakeholder management, and architecture work may see less benefit.
- **Study variation:** The 22–55% range is wide. Task type, codebase complexity, and team adoption rate all moderate the effect. Use the central estimate for planning; use the range for risk scenarios.

- **COCOMO TOOL interaction:** The TOOL driver already accounts for some tooling quality. Using both TOOL at Very High and a Heavy AI multiplier may double-count some benefit.

10.4 What the Model Cannot Do

- Predict individual sprint velocity or week-by-week burn rate
- Account for organisational dysfunction, communication failures, or political delays
- Estimate effort for research-phase work where requirements are undefined
- Replace expert estimation — it is a structured input to a human decision, not a replacement for one

11. The Mitacs Vision — Why Company Data Matters

Every model in this tool is limited by the data it was trained on. COCOMO-81 learned from 1980s defense software. The GitHub model learned from open-source volunteers. **Neither knows your team.**

A model trained on your company's completed projects — with actual Jira-tracked hours as the ground truth — would:

- Learn your team's specific velocity, not a 1981 average
- Identify which COCOMO drivers actually predict overruns *for you*
- Discover your chronically underestimated project types
- Quantify how much AI tool adoption is actually saving your team (not a study estimate)

11.1 What Can Be Claimed Today

Model	Training data	Validation MMRE	PRED(25)
COCOMO-81 benchmark	63 projects	0.3910 nested CV	46.0%
GitHub proof of concept	91 repos	0.2055 nested CV	70.3%
Your company data	Not yet provided	Not measured	Not measured

Company data could be more representative of the intended use case, but no numerical improvement is promised. Its MMRE and PRED(25) must be measured using held-out completed projects after the target definition and data quality are validated.

11.2 What You Would Provide

- **From Jira:** Story points, sprint velocity, cycle time, bug reopen rate, team size, planned vs actual delivery
- **From GitHub:** PRs merged, review time, test coverage, language count, AI tool adoption, commit frequency

- **Scale:** A small upload can be checked for schema and feasibility, but reliable model validation generally needs substantially more completed projects; uncertainty must be reported for the actual sample.

11.3 The 4-Month Deliverable

Phase	Timeline	Deliverable
Data collection & feature engineering	Month 1–2	Clean dataset with effort ground truth + feature set
Model training & validation	Month 3	FAHP-SVR model, SHAP integration, nested validation and chronological hold-out when dates permit
Deployment & handoff	Month 4	Live interactive dashboard on your internal infrastructure, documentation, maintenance guide — no sensitive data leaves your environment

11.4 Cost Structure

Contributor	Amount	What it covers
Industry partner	$7,500	4 months of focused research and development
Mitacs (federal funding)	$7,500	Matched 1:1 — equal federal and industry contribution
Total research value	$15,000	Custom AI estimation tool delivered in 4 months

Next step: Use the app's downloadable template and local validator. A feasibility review can assess completeness and target quality, but achievable MMRE is established only by leakage-free validation on the resulting dataset.

12. References

Reference	Relevance
Boehm, B.W. (1981). Software Engineering Economics. Prentice Hall.	COCOMO-81 model, cost drivers, EAF
Saaty, T.L. (1980). The Analytic Hierarchy Process. McGraw-Hill.	Foundation of AHP
Buckley, J.J. (1985). Fuzzy hierarchical analysis. Fuzzy Sets and Systems, 17(3), 233–247.	Fuzzy TFN scale for pairwise comparisons
Chang, D.Y. (1996). Applications of the extent analysis method on fuzzy AHP. European Journal of Operational Research, 95(3), 649–655.	Extent analysis algorithm — FAHP weights
Sehra, S.K., Brar, Y.S., Kaur, N. (2018). Multi-objective optimization for software effort estimation using FAHP and SVR. Soft Computing, 22(8).	Core methodology this tool implements
Peng, S. et al. (2023). The impact of AI coding assistants on developer productivity: A controlled experiment. GitHub Research.	55% faster task completion in the controlled experiment
McKinsey Global Institute (2023). The economic potential of generative AI.	20–45% developer productivity gain
Kalliamvakou, E. (2022). Research: Quantifying GitHub Copilot's impact on developer productivity. GitHub Blog.	Copilot PR merge speed study
Lundberg, S.M., Lee, S.I. (2017). A unified approach to interpreting model predictions. NeurIPS 30.	SHAP explainability framework
Stack Overflow Developer Survey (2024). stackoverflow.com/survey	AI tool adoption statistics
GitHub Octoverse (2024). github.com/octoverse	Copilot code contribution data